

Lógica para Computação – Respostas dos Exercícios

Exercícios 10.1 a 10.16 – Apostila de Introdução à Programação em Lógica (UFCG, 2006)

Exercício 10.1

Respostas para as consultas ao programa de relações familiares:

1. ?- genitor(X, jim).

X = pat

(Pat é a única genitor de Jim na base de fatos.)

2. ?- genitor(jim, X).

No

(Jim não aparece como genitor de ninguém na base de fatos.)

3. ?- genitor(pam, X), genitor(X, pat).

X = bob

(Pam $\rightarrow$ Bob $\rightarrow$ Pat: Pam é genitor de Bob, e Bob é genitor de Pat.)

4. ?- genitor(pam, X), genitor(X, Y), genitor(Y, jim).

X = bob, Y = pat

(Pam $\rightarrow$ Bob $\rightarrow$ Pat $\rightarrow$ Jim: cadeia de três gerações.)

Exercício 10.2

Consultas formuladas e seus resultados:

Quem são os pais de Pat?

?- genitor(X, pat).

X = bob

Bob é o único genitor de Pat.

Liz possui filhos?

?- genitor(liz, X).

No

Liz não aparece como genitor de ninguém no programa base (filhos adicionados no Ex. 10.3.6).

Quem é o avô de Pat?

?- avos(X, pat), homem(X).

X = tom

Tom $\rightarrow$ Bob $\rightarrow$ Pat. Usando homem(X) para filtrar apenas avôs masculinos.

Quem é a mãe de Jim?

?- mae(X, jim).

X = pat

Pat é genitor de Jim e mulher.

Quem é o irmão de Bob?

?- irmao(X, bob).

No

Bob não possui irmãos homens: Tom tem Bob e Liz (Liz é mulher); Pam tem apenas Bob.

Quem é a irmã de Pat?

```
?- irma(X, pat).  
X = ann
```

Ann e Pat são ambas filhas de Bob e ambas mulher.

Exercício 10.3

Novas regras implementadas (ver também trabalho.pl):

10.3.1 – Tio / Tia

```
tio(X, Z) :- genitor(Y, Z), irmao(X, Y).  
tia(X, Z) :- genitor(Y, Z), irma(X, Y).
```

X é tio/tia de Z se X for irmão/irmã de um dos pais de Z.

10.3.2 – Irmão

```
irmao(X, Y) :-  
    genitor(Z, X), genitor(Z, Y),  
    homem(X), \+(X = Y).
```

Análogo à regra irma/2, mas para indivíduos masculinos.

10.3.3 – Avós Paternos

```
avos_paternos(X, Z) :-  
    genitor(Pai, Z), homem(Pai),  
    genitor(X, Pai).
```

X é avô/avó paterno(a) de Z se for genitor do pai de Z.

10.3.4 – Avós Maternos

```
avos_maternos(X, Z) :-  
    genitor(Mae, Z), mulher(Mae),  
    genitor(X, Mae).
```

X é avô/avó materno(a) de Z se for genitor da mãe de Z.

10.3.5 – Ascendente (inverso de descendente)

```
ascendente(X, Z) :- genitor(Z, X).  
ascendente(X, Z) :- genitor(Y, X), ascendente(Y, Z).
```

ascendente(X, Z): Z é um ancestral de X. Inverso de descendente(Z, X): Z tem X como descendente.

10.3.6 – Primo / Prima

```
% Novos fatos adicionados:  
genitor(liz, carol). genitor(liz, mike).  
mulher(carol).      homem(mike).  
  
primo(X, Y) :-  
    genitor(PA, X), genitor(PB, Y),  
    \+(PA = PB),  
    (irmao(PA, PB) ; irma(PA, PB) ; irmao(PB, PA) ; irma(PB, PA)),  
    \+(X = Y).
```

X e Y são primos se seus pais forem irmãos entre si. Exemplos: primo(carol, ann), primo(mike, pat).

Exercício 10.4

Em Prolog, **todas as consultas** utilizam encadeamento para trás (backward chaining): o sistema parte do objetivo e busca provar os sub-objetivos até encontrar fatos que os satisfaçam. A diferença está na profundidade do encadeamento:

?- genitor(pam, bob).

Verifica diretamente um fato — encadeamento de 1 passo. Sim, usa backward chaining.

?- mae(pam, bob).

Aplica a regra mae/2, provando genitor(pam,bob) e mulher(pam) — encadeamento por regra (múltiplos passos).

?- avos(pam, ann).

Aplica a regra avos/2, provando genitor(pam,bob) e genitor(bob,ann) — encadeamento multi-passo.

?- avos(bob, jim).

Aplica avos/2: genitor(bob,Y) → Y=pat, depois genitor(pat,jim) — encadeamento multi-passo.

Todas as quatro consultas utilizam encadeamento para trás. As consultas 2, 3 e 4 exigem encadeamento *através de regras*, enquanto a consulta 1 resolve-se por correspondência direta com um fato.

Exercício 10.5

Base de conhecimento animal — fatos e regras (ver trabalho.pl). Respostas às consultas solicitadas:

O piupiu voa?

?- pode_voar(piupiu).
Yes

piupiu é canário → pássaro → pode_voar(X) :- passaro(X), \+(nao_voa(X)). nao_voa/1 não se aplica a canários.

Qual a cor do piupiu?

?- cor(piupiu, C).
C = amarelo

cor(X, amarelo) :- canario(X). piupiu é canário.

A Xica voa?

?- pode_voar(xica).
No

xica é avestruz → nao_voa(xica) é verdade → pode_voar(xica) falha.

A Xica tem asas?

?- tem_asas(xica).
Yes

tem_asas(X) :- passaro(X). xica é avestruz → pássaro.

O Vamp voa?

?- pode_voar(vamp).
Yes

pode_voar(X) :- morcego(X). vamp é morcego.

O Vamp tem asas?

?- tem_asas(vamp).
Yes

tem_asas(X) :- morcego(X).

O Vamp põe ovos?

?- poe_ovos(vamp).
No

vamp é mamífero; poe_ovos só vale para pássaros e peixes (exceto tubarões).

Quais animais põem ovos?

```
?- poe_ovos(X).  
X = piupiu ; X = xica ; X = nemo ; X = alfred
```

piupiu e xica (pássaros); nemo e alfred (peixes sem nao_poe_ovos). tutu (tubarão) tem nao_poe_ovos(tutu).

Quais animais são comestíveis?

```
?- comestivel(X).  
X = mimosa ; X = alfred
```

mimosa é vaca (comestivel :- vaca); alfred é salmão (comestivel :- salmao).

Quais animais se movem andando?

```
?- anda(X).  
X = mimosa ; X = xica
```

mimosa: mamífero que não voa. xica: avestruz (regra específica).

Quais animais nadam mas não põem ovos?

```
?- pode_nadar(X), \+(poe_ovos(X)).  
X = tutu
```

tutu é tubarão → pode_nadar; mas nao_poe_ovos(tutu) → poe_ovos(tutu) falha.

Exercício 10.6

Respostas para as consultas ao programa de cadeia alimentar:

?- come(urso, peixinho).

No

Não existe o fato come(urso, peixinho). Urso come peixe, e peixe come peixinho, mas não há transitividade automática.

?- come(raposa, coelho).

Yes

Fato direto na base.

?- come(guaxinim, X).

X = peixe

O único fato com come(guaxinim, _) é come(guaxinim, peixe).

?- come(X, grama).

X = coelho ; X = veado

come(coelho, grama) e come(veado, grama) são fatos da base.

?- come(urso, X), come(X, coelho).

X = raposa

Urso come: peixe, guaxinim, raposa, veado. Desses, apenas raposa come coelho.

?- presa(X), not(come(raposa, X)).

X = peixe ; X = peixinho ; X = guaxinim ; X = raposa ; X = veado

Presas (animais comidos por outros): peixe, peixinho, guaxinim, raposa, coelho, veado. Raposa come apenas coelho, então coelho é excluído pelo not.

Exercício 10.7

Regras para herbívoro e carnívoro:

```
herbivoro(X) :- animal(X), come(X, Y), planta(Y).
```

```
carnivoro(X) :- animal(X), come(X, Y), animal(Y).
```

Um animal é herbívoro se come algo que é planta; é carnívoro se come algo que é animal.

Exercício 10.8

Consultas usando as regras do Exercício 10.7:

Quais animais são herbívoros?

```
?- herbivoro(X).
```

```
X = peixinho ; X = coelho ; X = veado
```

peixinho come alga (planta); coelho e veado comem grama (planta).

Quais animais são carnívoros?

```
?- carnivoro(X).
```

```
X = urso ; X = peixe ; X = guaxinim ; X = raposa ; X = lince
```

Todos comem pelo menos um outro animal.

Quais herbívoros são presas de uma raposa?

```
?- herbivoro(X), come(raposa, X).
```

```
X = coelho
```

Raposa come apenas coelho; coelho é herbívoro.

Exercício 10.9

Programa com as capitais de todos os 26 estados + DF (ver trabalho.pl). Exemplo de consulta:

```
?- capital_do_estado(minas_gerais, C).
```

```
C = belo_horizonte
```

```
?- capital_do_estado(E, brasilia).
```

```
E = distrito_federal
```

```
?- capital_do_estado(E, C). % lista todos
```

```
E = acre, C = rio_branco ;
```

```
E = alagoas, C = maceio ; ...
```

O predicado `capital_do_estado/2` simplesmente delega para `capital/2`, servindo como interface de consulta.

Exercício 10.10

Compatibilidade de tipos sanguíneos (ver trabalho.pl):

```
pode_doar(o, o). pode_doar(o, a). pode_doar(o, b). pode_doar(o, ab).
```

```
pode_doar(a, a). pode_doar(a, ab).
```

```
pode_doar(b, b). pode_doar(b, ab).
```

```
pode_doar(ab, ab).
```

```
pode_receber(Receptor, Doador) :- pode_doar(Doador, Receptor).
```

Exemplos de consultas:

```
?- pode_doar(o, X). % O doa para quem?
```

```
X = o ; X = a ; X = b ; X = ab
```

```
?- pode_receber(ab, X). % AB recebe de quem?
```

```
X = o ; X = a ; X = b ; X = ab
```

```
?- pode_receber(a, X). % A recebe de quem?
```

```
X = o ; X = a
```

O tipo O é o doador universal; o tipo AB é o receptor universal.

Exercício 10.11

Predicados sobre listas (implementações em trabalho.pl):

10.11.1 – Interseção de duas listas

```
intersecao([], _, []).
intersecao([H|T], L2, [H|I]) :- member(H, L2), !, intersecao(T, L2, I).
intersecao(_|T, L2, I)      :- intersecao(T, L2, I).

?- intersecao([1,2,3,4], [2,4,6], X).
X = [2, 4]
```

10.11.2 – Subconjunto contido em lista

```
subconjunto([], _).
subconjunto([H|T], L) :- member(H, L), subconjunto(T, L).

?- subconjunto([1,3], [1,2,3,4]).
Yes
```

10.11.3 – Tamanho par ou ímpar

```
tamanho_par([]).
tamanho_par(_|_|T) :- tamanho_par(T).

tamanho_impar([]).
tamanho_impar(_|_|T) :- tamanho_impar(T).

?- tamanho_par([1,2,3,4]).    Yes
?- tamanho_impar([1,2,3]).    Yes
```

10.11.4 – Lista invertida usando concatenação

```
inverter([], []).
inverter([H|T], R) :- inverter(T, TI), append(TI, [H], R).

?- inverter([1,2,3], R).
R = [3, 2, 1]
```

10.11.5 – Maior valor em lista numérica

```
maximo([X], X).
maximo([H|T], M) :- maximo(T, M1), M is max(H, M1).

?- maximo([3,1,7,2,5], M).
M = 7
```

10.11.6 – Soma dos N primeiros números naturais

```
soma_naturais(0, 0) :- !.
soma_naturais(N, S) :-
    N > 0, N1 is N - 1,
    soma_naturais(N1, S1), S is S1 + N.

?- soma_naturais(5, S).
S = 15    % 1+2+3+4+5 = 15
```

Exercício 10.12

N-ésima potência de um número:

```
potencia(_, 0, 1) :- !.  
potencia(Base, Exp, Result) :-  
    Exp > 0,  
    Exp1 is Exp - 1,  
    potencia(Base, Exp1, R1),  
    Result is Base * R1.  
  
?- potencia(2, 10, R).    R = 1024  
?- potencia(3, 4, R).    R = 81  
?- potencia(5, 0, R).    R = 1
```

Base: qualquer número elevado a 0 é 1 (corte evita retrocesso). Passo recursivo: $\text{Base}^{\text{Exp}} = \text{Base} * \text{Base}^{(\text{Exp}-1)}$.

Exercício 10.13

Versão eficiente de checagem usando cortes:

```
checagem(N, positivo) :- N > 0, !.  
checagem(0, zero)      :- !.  
checagem(_, negativo).
```

O corte (!) após a condição de guarda elimina as alternativas restantes assim que uma cláusula é selecionada. No programa original, se $N=5$, Prolog provaria $N>0$ (verdadeiro), retornaria positivo — mas, sem corte, continuaria a tentar as cláusulas seguintes ao retroceder. Com o corte, após confirmar $N>0$ e produzir "positivo", nenhuma outra cláusula é tentada.

Exercício 10.14

Classificação por faixa etária com cortes:

```
classificar(Idade, crianca)      :- Idade <= 12, !.  
classificar(Idade, adolescente) :- Idade <= 18, !.  
classificar(Idade, adulto)      :- Idade <= 65, !.  
classificar(_, idoso).  
  
?- classificar(8, C).    C = crianca  
?- classificar(15, C).  C = adolescente  
?- classificar(30, C).  C = adulto  
?- classificar(70, C).  C = idoso
```

O corte garante que, uma vez encontrada a categoria correta, Prolog não tentará as cláusulas seguintes. A última cláusula não precisa de condição nem de corte, pois só é alcançada quando todas as anteriores falharam.

Exercício 10.15

Signo zodiacal com cortes — signo(+Dia, +Mes, -Signo):

```
signo(D, 12, capricornio) :- D >= 22, !  
signo(D, 1, capricornio) :- D <= 19, !  
signo(D, 1, aquario)      :- D >= 20, !  
signo(D, 2, aquario)      :- D <= 18, !  
... (demais signos em ordem – ver trabalho.pl)  
signo(_, 12, sagitario).
```

O predicado trata cada par (mês, intervalo de dias) como uma cláusula separada. O corte garante que, ao encontrar o signo correto, Prolog não tente outras cláusulas. Capricórnio é o único signo que cruza a virada do ano (22/12 – 19/01), por isso aparece em duas cláusulas separadas.

Exemplos:

```
?- signo(25, 12, S).    S = capricornio
?- signo(15, 12, S).    S = sagitario
?- signo(10, 1, S).     S = capricornio
?- signo(25, 6, S).     S = cancer
?- signo(1, 3, S).      S = peixes
```

Datas utilizadas para cada signo:

- Capricórnio: 22/12 – 19/01
- Aquário: 20/01 – 18/02
- Peixes: 19/02 – 20/03
- Áries: 21/03 – 19/04
- Touro: 20/04 – 20/05
- Gêmeos: 21/05 – 20/06
- Câncer: 21/06 – 22/07
- Leão: 23/07 – 22/08
- Virgem: 23/08 – 22/09
- Libra: 23/09 – 22/10
- Escorpião: 23/10 – 21/11
- Sagitário: 22/11 – 21/12

Exercício 10.16

Melhoria do código de tradução de erros:

Versão original (procedimental):

```
traduza(Codigo, Sig) :- Codigo=1, Sig=integer_overflow.
traduza(Codigo, Sig) :- Codigo=2, Sig=divisao_por_zero.
traduza(Codigo, Sig) :- Codigo=3, Sig=id_desconhecido.
```

Versão melhorada (declarativa):

```
traduza(1, integer_overflow).
traduza(2, divisao_por_zero).
traduza(3, id_desconhecido).
```

A versão original usa unificação no corpo das cláusulas (`Codigo=1`, `Sig=valor`), o que é um estilo procedural desnecessário. A versão melhorada declara o relacionamento diretamente como *fatos* : o padrão de unificação acontece na cabeça da cláusula, que é exatamente como Prolog funciona naturalmente. O resultado é código mais legível, mais conciso e mais fácil de estender.

Adicionalmente, a versão declarativa permite consultas inversas: `?- traduza(C, divisao_por_zero).` $\rightarrow C = 2$, o que não é possível de maneira direta com a versão original.